

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 1

The OS as a Resource Manager

Track A: the real OS, from the outside (Linux)

Course: **Operating Systems**

Author: Rotaru Anastasia
Group: FAF-242
Checked by: Reițman Patricia

Chişinău, 2026

Contents

1 Goal of the work **2**
1.1 Working environment . . . 2

2 Part 1. Files and directories **2**
2.1 Navigating and inspecting . . . 2
2.2 Creating, copying, renaming and removing . . . 3
2.3 Ownership and permissions . . . 3
2.4 Observations . . . 3

3 Part 2. Processes **4**
3.1 Listing processes . . . 4
3.2 A background process: start, find, stop . . . 4
3.3 The `/proc` pseudo-filesystem . . . 5
3.4 Observations . . . 5

4 Part 3. Memory **6**
4.1 Observations . . . 6

5 Part 4. Devices and storage **7**
5.1 Observations . . . 7

6 Conclusion **8**

1 Goal of the work

The operating system (OS) is the manager that stands between user programs and the hardware. In this lab I observed, in the terminal of a Linux virtual machine, the four core jobs of any OS: managing **files**, **processes**, **memory** and **devices**. For every part I ran the commands given in the assignment, included the output I obtained, and answered the observation questions in my own words.

1.1 Working environment

```
mkdir -p ~/os-lab1 && cd ~/os-lab1
whoami        # the current user
uname -a      # the kernel managing this machine
uptime        # how long the OS has been running
```

```
vboxuser
Linux UbuntuLab 7.0.0-34-generic #34-Ubuntu SMP PREEMPT_DYNAMIC Wed Sep  2 14:34:54
    UTC 2026 aarch64 GNU/Linux
 19:51:51 up 31 min,  1 user,  load average: 0.42, 0.24, 0.21
```

The lab was done on an Ubuntu virtual machine (hostname `UbuntuLab`) running under VirtualBox, logged in as the user `vboxuser`. The kernel is Linux 7.0.0 built for the `aarch64` (ARM64) architecture, and the system had been up for 31 minutes.

2 Part 1. Files and directories

The OS turns a raw disk into a tidy tree of named files, each with an owner and permissions. In this part I explored that tree and changed it.

2.1 Navigating and inspecting

```
pwd                 # where am I
ls -la /            # the root of the whole tree
ls -la ~            # my home directory
cd /etc && ls | head
```

```
/home/vboxuser/os-lab1
total 80
drwxr-xr-x  20 root root  4096 Sep 28 13:29 .
lrwxrwxrwx   1 root root     7 Apr 20 08:46 bin -> usr/bin
drwxr-xr-x   4 root root  4096 Sep 28 13:36 boot
drwxr-xr-x  18 root root  4080 Sep 28 19:20 dev
drwxr-xr-x 142 root root 12288 Sep 28 13:39 etc
drwxr-xr-x   3 root root  4096 Sep 28 13:33 home
dr-xr-xr-x 272 root root     0 Sep 28 19:20 proc
drwxr-xr-x  11 root root  4096 Aug 26 09:14 usr
drwxr-xr-x  14 root root  4096 Sep 28 13:37 var
```

The command `pwd` prints the absolute path of the current directory. The command `ls -la` lists everything in a directory, including hidden files, in long format. In the listing

of the root I noticed that `/bin` is not a real directory but a symbolic link (the leading `l`) pointing to `usr/bin`.

2.2 Creating, copying, renaming and removing

```
cd ~/os-lab1
mkdir demo && cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
```

```
mkdir: demo: File exists
total 16
drwxrwxr-x 2 vboxuser vboxuser 4096 Sep 28 14:20 demo
-rw-rw-r-- 1 vboxuser vboxuser   24 Sep 28 19:53 note.txt
-rw-rw-r-- 1 vboxuser vboxuser   24 Sep 28 19:53 renamed.txt
-rw-rw-r-- 1 vboxuser vboxuser  808 Sep 28 14:50 report.md
```

`mkdir` creates a directory, `echo ... >` writes text into a file, `cp` copies it, `mv` renames (or moves) it and `rm` deletes it. The folder `demo` already existed from an earlier attempt, so `mkdir` reported an error, and because the commands are joined with `&&`, the `cd demo` did not run. As a result the files were created directly in `~/os-lab1` instead of `demo`. This also shows that the OS refuses to create a second directory with the same name in the same place.

2.3 Ownership and permissions

```
ls -l note.txt
chmod 600 note.txt   # owner-only read+write
ls -l note.txt
chmod 644 note.txt
```

```
-rw-rw-r-- 1 vboxuser vboxuser 24 Sep 28 19:53 note.txt
-rw------- 1 vboxuser vboxuser 24 Sep 28 19:53 note.txt
```

The first line is the state before `chmod`, the second one after `chmod 600`: the group and the others lost their rights.

2.4 Observations

1. Who owns the files I created, and which group?
The files are owned by the user `vboxuser`, the account I am logged in with, and their group is also `vboxuser`. On Ubuntu every user gets a private group with the same name as the user, and the files he creates belong to it by default. The OS stores both the owner and the group in the metadata of every file and uses them to decide who may access it. The default permissions of a new file were `-rw-rw-r-`.

2. What do the ten characters at the start of `ls -l` mean after `chmod 600`?
After `chmod 600` the string is `-rw----`. The first character describes the type of the entry:

a dash means a regular file (a `d` would mean a directory and an `l` a symbolic link). The next three characters, `rw-`, are the permissions of the owner, who may read and write but not execute. The following three, `--`, are the permissions of the group, and the last three, `--`, are those of everyone else; both have no rights at all. The number 600 encodes the same thing, because each digit is a sum of read (4), write (2) and execute (1), so 6 is 4+2, that is read and write, and 0 is no permission.

3. Two directories directly under `/`.
The directory `/etc` holds the system-wide configuration files of the machine and of the installed programs. The directory `/home` holds the personal directories of the users; mine is `/home/vboxuser`, where my files and settings are stored.

3 Part 2. Processes

Every running program is a process that the OS creates, isolates and schedules. In this part I listed processes, started one in the background and stopped it.

3.1 Listing processes

```
ps aux | head
ps aux | wc -l      # how many processes exist right now
top                 # live view; press q to quit
```

```
USER   PID %CPU %MEM   VSZ   RSS TTY STAT START TIME COMMAND
root     1  0.0  0.3 27480 10952 ?   Ss   19:20 0:00 /usr/lib/systemd/systemd --
    switched-root --system --deserialize=51
root     2  0.0  0.0     0     0 ?   S    19:20 0:00 [kthreadd]
root     3  0.0  0.0     0     0 ?   S    19:20 0:00 [pool_workqueue_release]
root     4  0.0  0.0     0     0 ?   I<   19:20 0:00 [kworker/R-rcu_gp]
...
213

Tasks: 211 total,   3 running, 207 sleeping,   0 stopped,   1 zombie
%Cpu(s):  6.0 us,  2.9 sy,  0.0 ni, 90.1 id,  0.5 wa
MiB Mem :   3379.0 total,     67.6 free,   3021.8 used,    451.7 buff/cache
```

`ps aux` prints one line per process (user, PID, CPU and memory usage, state, command). `wc -l` counts the lines, which gives the number of processes plus one header line. `top` shows the same information updated live. Names in square brackets, such as `[kthreadd]`, are threads of the kernel itself.

3.2 A background process: start, find, stop

```
sleep 300 &          # run in the background; note the PID printed
jobs                 # shell view of background jobs
ps -ef | grep sleep  # find it among all processes
kill <PID>           # ask the OS to terminate it
```

```
[1] 7025
[1]+  Running                 sleep 300 &
```

```
vboxuser    7025    6908  0 19:54 pts/0    00:00:00 sleep 300
vboxuser    7027    6908  0 19:54 pts/0    00:00:00 grep --color=auto sleep
```

The `&` sign starts the command in the background so that the terminal stays free. The shell printed the PID `7025`, the number the OS assigned to the new process. In the output of `ps -ef` the second number, `6908`, is the PID of the parent process, which is my shell. The process was then stopped with `kill 7025`, which sends it a signal (by default `SIGTERM`) asking it to terminate. Note that `<PID>` in the assignment is a placeholder to be replaced by the real number, not text to type.

3.3 The `/proc` pseudo-filesystem

```
sleep 300 &
PID=$!
ls /proc/$PID/
cat /proc/$PID/status | head -20   # state, memory, threads
kill $PID
```

```
[2] 7034
attr    cmdline  environ  fd  limits  maps  mem  status  task ...
Name:   sleep
Umask:  0002
State:  S (sleeping)
Tgid:   7034
Pid:    7034
PPid:   6908
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
VmPeak:    15480 kB
VmSize:    15480 kB
[2]+  Terminated                 sleep 300
```

`/proc` does not exist on the disk. The kernel generates it on the fly to show information about itself and about every process. The variable `$!` holds the PID of the last background process, and the last line confirms that `kill` terminated it.

3.4 Observations

1. What is the PID of process number 1, and what is it?
Process number 1 has PID 1 and its command is `/usr/lib/systemd/systemd`, so it is `systemd`, the init system of Ubuntu. It is the first process started by the kernel at boot, every other process descends from it, and it starts and supervises the system services.

2. Roughly how many processes were running on the idle VM?
The command `ps aux | wc -l` printed 213, which is 212 processes after removing the header line, and `top` reported 211 tasks (3 running, 207 sleeping and 1 zombie). The two numbers differ slightly because they were taken at different moments and `ps` itself is a process. So there were roughly 210 processes. Almost all of them were sleeping: kernel threads, system services and the desktop programs (`gnome-shell`, `firefox`). This shows that the OS keeps track of a very large number of processes even when the user does nothing.

3. What does the `State:` line say for the sleeping process?
The line reads `S (sleeping)`. It means the process is not using the CPU: it is waiting for an event (here, the timer of 300 seconds). The scheduler ignores it until the event happens, and then the kernel wakes it up and lets it finish. The line `PPid: 6908` also confirms that its parent is my shell.

4 Part 3. Memory

The OS gives each process its own view of memory and shares the physical RAM between all processes.

```
free -h                        # total / used / free RAM and swap
cat /proc/meminfo | head -6
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status  # real RAM used by this process
kill $PID
```

```
               total        used        free      shared  buff/cache   available
Mem:           3.3Gi       2.9Gi        99Mi        45Mi       460Mi       403Mi
Swap:             0B          0B          0B
MemTotal:        3460128 kB
MemFree:           97244 kB
MemAvailable:     408724 kB
Buffers:            3368 kB
Cached:           431880 kB
SwapCached:            0 kB
VmRSS:        16 kB
```

`free -h` shows the memory totals in human-readable units. `VmRSS` (resident set size) is the amount of physical RAM that the process really occupies right now.

4.1 Observations

1. How much total RAM does the VM have, and how much is free?
According to `free -h`, the VM has 3.3 GiB of RAM in total (`MemTotal` is 3 460 128 kB), of which only 99 MiB is free. This looks alarming, but the more important column is *available*, 403 MiB, which counts the 460 MiB of cache that the OS can give back to programs when they need it. Most of the 2.9 GiB used is taken by the desktop environment and the web browser. The OS deliberately uses spare RAM as cache, so "free" memory is usually small.

2. What is swap, and how much is configured?
Swap is an area on the disk that the OS uses as an extension of RAM. When physical memory runs low, the OS moves pages that are not used at the moment out to swap, which frees RAM for active processes. Swap is much slower than RAM, so it is only a safety net. On my VM the `Swap:` line shows `0B`, so no swap is configured. With little available RAM and no swap, the system has nowhere to move pages to and would have to terminate programs if memory ran out completely.

3. How much resident memory does a bare `sleep` use, and does it surprise me?
The `sleep` process has `VmRSS` of only 16 kB. It was a bit surprising to see how small

this is, especially compared with its virtual size `VmSize` of 15 480 kB, about 15 MB. The difference shows that the OS gives a process a large virtual address space but assigns real RAM only for the pages that are actually used. `sleep` just waits for a timer and stores almost no data, so it needs very little.

5 Part 4. Devices and storage

The OS hides every disk, terminal and device behind uniform, file-like interfaces. This is the UNIX idea that "everything is a file".

```
df -h                 # mounted filesystems and free space
lsblk                 # the block devices (disks) themselves
du -sh ~/os-lab1      # size of my work folder
ls -l /dev | head     # devices presented as files
mount | head
```

```
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           676M  2.3M  674M   1% /run
/dev/sda2        24G  7.2G   16G  33% /
/dev/sda1       1.1G  6.6M  1.1G   1% /boot/efi
/dev/sr0         51M   51M     0 100% /run/media/vboxuser/VBox_GAs_7.2.20

NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
sda      8:0    0    25G  0 disk
|-sda1   8:1    0     1G  0 part /boot/efi
‘-sda2   8:2    0  23.9G  0 part /
sr0     11:0    1  50.7M  1 rom  /run/media/vboxuser/VBox_GAs_7.2.20

20K     /home/vboxuser/os-lab1

crw-r--r--  1 root root  10, 235 Sep 28 19:20 autofs
drwxr-xr-x  2 root root      400 Sep 28 19:27 block
lrwxrwxrwx+ 1 root root        3 Sep 28 19:20 cdrom -> sr0
crw-------  1 root tty    5,   1 Sep 28 19:20 console

/dev/sda2 on / type ext4 (rw,relatime)
```

In the output of `df -h`, the column *Filesystem* shows the device and the column *Mounted on* shows where it is attached in the file tree. `lsblk` shows the disks and their partitions. In `ls -l /dev`, a first character `b` marks a block device (such as a disk) and `c` marks a character device (such as a terminal); `console` above is a character device. My work folder takes only 20 kB.

5.1 Observations

1. Which device is the root filesystem `/` mounted on?
The root filesystem is mounted on `/dev/sda2`, which is confirmed by the output of `mount` (`/dev/sda2 on / type ext4`). It is the second partition of the 25 GB virtual disk `sda`, it uses the `ext4` filesystem, has a size of 24 GB and is 33% full. The first partition, `/dev/sda1`, is mounted on `/boot/efi` and is used for booting.

2. One entry from `/dev` and the real thing it stands for.

The entry `/dev/sda` stands for the whole virtual hard disk of the machine, and `/dev/sda1` and `/dev/sda2` for its partitions. Another example is `/dev/cdrom`, a link to `/dev/sr0`, which stands for the virtual optical drive that holds the VirtualBox Guest Additions image.

3. What does "everything is a file" mean?
It means that the OS presents disks, terminals and other devices as entries in the file tree, so a program can read from and write to them with the same operations it uses for ordinary files, without knowing the details of the hardware. I saw the same idea in Part 2, where even information about processes was available as files under `/proc`.

6 Conclusion

The operating system manages four core resources: files, processes, memory and devices. I observed the files with `ls -l` and `chmod`, which showed the tree, the owners and the permissions, and the processes with `ps aux` and `/proc/<pid>/status`, which showed what is running and in what state. I observed the memory with `free -h`, which shows RAM and swap, and the devices with `df -h` and `ls -l /dev`, which show the disks mounted in the file tree and the device files.

This lab showed me that the OS is not an abstract idea: I could see its work directly, in the file tree, in about 210 processes running on an apparently idle machine, in the memory counters and in the device files. The main lesson is that the OS gives every program a simple, uniform view of complicated hardware and shares the limited resources between all of them. For example, a 16 kB `sleep` process has a 15 MB virtual address space, and the root disk `/dev/sda2` is used exactly like a folder in the file tree.